Matrix Multiplication Benchmark Report

Anna Sowińska

October 2025

1 Objective

The goal of this project was to compare the execution performance of a simple matrix multiplication algorithm implemented in three programming languages: **C++**, **Java**, and **Python**. All implementations were designed following the principles of:

- Separation of production code from testing code,
- Several runs for each experiment (averaging),
- Parametrization of the matrix calculation through command line arguments.

2 Implementation Overview

Each implementation consists of two main parts:

- **Production code** — function `matmul()` responsible only for matrix multiplication.
- **Benchmark code** — functions performing repeated tests, timing, and printing results.

C++

Implemented in `matrix.cpp` using `std::vector` for memory management and `chrono` for timing.

Java

Implemented in `Matrix.java`, using `System.nanoTime()` for precise measurements and arrays for data storage.

Python

Implemented in `matrix.py`, using lists of lists and `time.perf_counter()` for timing. Despite Python's simplicity, it is significantly slower due to the interpreted nature of the language.

3 Methodology

Each program was run for several matrix sizes: 50, 100, 200, and 500. For each size, three runs were executed, and the average execution time was computed.

All experiments were conducted on the same machine under similar conditions.

4 Results

The following tables summarize the average times measured in seconds:

C++

Size	Average Time (s)
50	0.000106
100	0.001016
200	0.008949
500	0.185643

Java

Size	Average Time (s)
50	0.000917
100	0.001249
200	0.011002
500	0.175167

Python

Size	Average Time (s)
50	0.008322
100	0.058834
200	0.506840
500	10.165894

5 Analysis and Discussion

- The results clearly show that **C++** is the fastest language for this task, followed closely by **Java**.
- **Python** is significantly slower (over 50x) for larger matrices due to dynamic typing and interpreter overhead.
- All implementations show roughly cubic time complexity ($O(n^3)$) as expected.
- The benchmarking methodology ensured repeatable results through multiple runs and averaging.

6 Conclusions

- Separating production and benchmarking code improves code readability and modularity.
- Parametrization allowed for flexible testing of different matrix sizes.
- Compiled languages (C++ and Java) are much more efficient for numerical computations than interpreted ones (Python).

7 Repository

The full project (source code, results, and report) is available at: `https://github.com/asowinska9/mat`